TLS-D: An integrated design of TLS protocol with decentralized identification

Xiaotong Chen[†,§], Linfu Sun[†,§,*], and Tong Gu[†,§]
[†]*School of Computing and Artificial Intelligence, Southwest Jiaotong University, Chengdu, China*
[§]*Manufacturing Industry Chain Collaboration Industrial Software Key Laboratory of Sichuan Province, Southwest Jiaotong University, Chengdu, China*
c0672@my.swjtu.edu.cn, sunlf@163.vip.com, gutong@my.swjtu.edu.cn

2025 11th International Conference on Computer and Communications (ICCC) | 979-8-3315-4558-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCC68654.2025.11437625

***Abstract*—Transport Layer Security Protocol 1.3 (TLS 1.3) improves security and performance by improving key exchange and simplifying the handshake process. However, (i) the multiple iterations computation required for temporary key exchange can lead to response latency issues, (ii) it suffers from a single point of trust with the root certificate authority (CA). To overcome this issue, this paper proposes a novel approach to the TLS-D authentication scheme. The framework enables customers to autonomously register and manage their identities through the blockchain, eliminating the reliance on centralized CAs. In addition, it transfers the authentication overhead to the distributed nodes to share while ensuring tamper-proof identity records. Distributed key generation further distributes key generation tasks to multiple nodes, enhancing fault tolerance, and reducing single-point computational bottlenecks. The experimental results show that TLS-D achieves dramatic latency reductions of up to 98.9% compared to TLS 1.3, while eliminating extreme handshake delays under 0-RTT replay attack conditions. It meets performance standards and security requirements.**

***Index Terms*—TLS 1.3, Certificate authority, Decentralized identity, Distributed key generation**

I. Introduction

TLS 1.3 pioneers an advanced secure transport paradigm that ensures data integrity and confidentiality in unreliable network environments through encrypted communication [1]. Significantly optimized in terms of handshake processes and encryption algorithms compared to legacy TLS versions, TLS 1.3 provides improved security and performance [2]. As a fundamental building block of Internet security [3], TLS 1.3 can be defined as a cryptographic protocol that ensures the privacy and authenticity of data during network communication [4]. TLS 1.3 plays an indispensable role in protecting sensitive information, preventing man-in-the-middle attacks, and ensuring the security of data transfers, and significantly improves network security and safety.

A bevy of literature [5]–[11] studies the current stage of TLS 1.3, which can primarily be categorized as: *(i) Performance improvements.* Chen et al. [5] compare the security and availability attributes of the protocols and compare the reliability, flow control, and congestion control of the layered protocols to better understand the benefits and limitations of secure channel establishment protocols. Cremers et al. [6] not only provide the first supporting evidence for the security of several complex protocol-mode interactions in TLS 1.3, but also show the strict necessity of the latest proposal to include more information in the content of protocol signatures. Dowling et al. [7] analyzed the complete TLS 1.3 handshake protocol, including the standard handshake and a pre-shared key-based model, using a multistage key exchange security model to demonstrate that they are able to establish session keys with the required security properties under standard cryptographic assumptions. *(ii) Security optimization* Platon et al. [8] analyzed how the TLS ecosystem responds to high-profile security attacks. Cremers et al. [9] provide an in-depth analysis of the security features and potential vulnerabilities of TLS 1.3, demonstrating its improvements over previous versions. Lee et al. [10] explored the practical deployment of TLS 1.3 by analyzing 687M connections in a temporal, spatial, and platform-based approach, revealing significant improvements in security and performance. Karthikeyan et al. [11] developed a symbolic and computational model for TLS 1.3 by proposing a methodology, combined with a high-assurance reference implementation, that comprehensively analyses its security, verifies its ability to protect against known attacks, and identifies potential vulnerabilities that affect the protocol design. While TLS 1.3 facilitates the efficiency of a centralized certificate authority and improves security, performance, and resilience to decentralization, it still encounters several challenges in practical implementations: ① computational costs, and ② single-point trust risk.

To overcome the aforementioned challenges, we propose a novel TLS authentication scheme, named TLS-D, which reduces the reliance on traditional centralized certificate authorities. It guarantees user autonomy as well as significantly improves response efficiency. The credibility and security of authentication are enhanced by using the decentralized nature of decentralized identity (DID) technology. A distributed key generation mechanism approach is considered for node screening of issuers.

Contributions. Our contributions can be summarized as:

- We propose an effective approach of the TLS authentication scheme TLS-D for performance tuning of TLS 1.3, which constructs a multi-objective integration framework that balances flexibility, interoperability, and privacy protection in the process of ensuring security and overall system efficiency.
- We implement the registration of decentralized identifier using a new authentication method that considers inter-

action with the blockchain. Considering the adoption of distributed key generation to select highly trusted and resourceful nodes for issuer identity holders.

- Experimental results show that TLS-D achieves latency reductions in handshake times compared to TLS 1.3, delivering speedups of up to 92.5×, while eliminating extreme latency spikes. Our test on 0-RTT replay attacks also verified a sustained improvement in performance, with average latency reductions of 97-98.7% across different connectivity scenarios. DID extensions provide strong authentication capabilities, while certificate size optimization reduces transport overhead while preserving verification integrity.

II. Related Work

A. Transport Layer Security (TLS)

TLS provides a secure channel between the client and the server in an Internet environment. This secure channel ensures the confidentiality and integrity of messages in transit, as well as the authentication of the server. The TLS 1.3 Handshake Protocol ensures the establishment of a secure channel by exchanging as shown in Fig. 1. **i) Key exchange.** The client sends a "ClientHello" message with key sharing parameters; the server replies with a "ServerHello" message, selects a cryptographic suite, and returns its key sharing parameters; both parties use these to generate a shared key. **ii) Server parameters.** The server sends an "EncryptedExtensions" message with protocol parameters, followed by key exchange parameters, and its certificate chain for client validation. **iii) Authentication.** The client verifies the server's certificate; both parties use the shared key to calculate the handshake key; both send a "Finished" message to confirm the handshake's integrity and success.

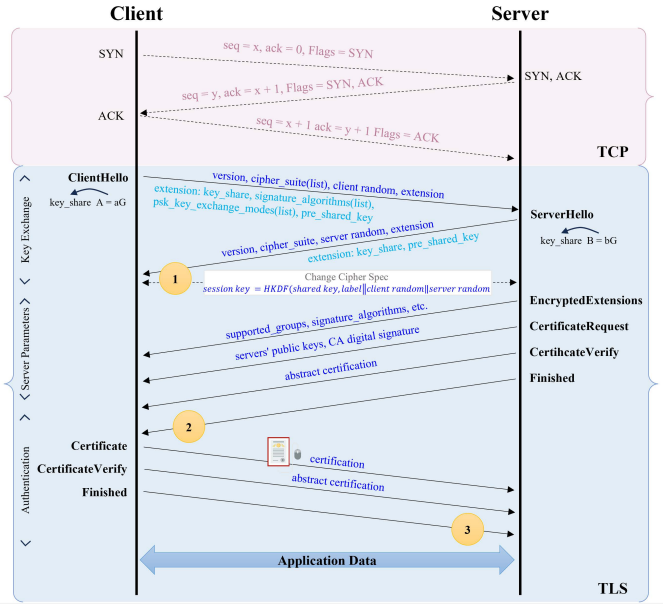

Fig. 1. Message flow in the original TLS 1.3 handshake protocol.

B. Decentralized Identity (DID)

Blockchain offers significant advantages for identifier registration systems. First, its decentralized architecture reduces reliance on a single authority and enhances system transparency and reliability [12]. Second, cryptographic and consensus mechanisms ensure data immutability, boosting system security [13], [14]. Additionally, blockchain's censorship resistance prevents manipulation by any single entity, ensuring fair identifier management [15].

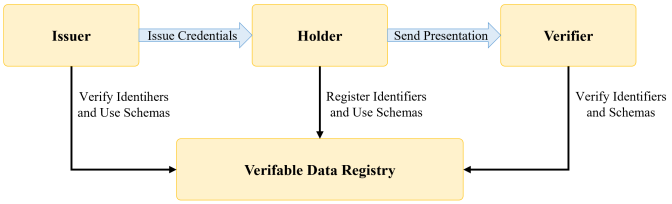

Fig. 2. Decentralized identity roles.

Decentralized identity [16] is a technology that enhances users' control over their identity data through decentralization, as shown in Fig. 2.

- *Decentralized identifier.* A globally unique decentralized identifier, such as $did : DID\_Method : 123...abc...$, consists of three parts: a fixed "did" prefix, a "DID Method" specifying the interaction mechanism with a specific blockchain, and an identifier unique to that DID method.
- *DID document object (DDO).* Each DID is linked to a DID Document, which includes the DID, associated public keys (e.g., for authentication), and other metadata. A DID can be resolved to obtain its DDO through methods defined by the corresponding DID method.
- *Claim.* Claim C contains the attribute A and the corresponding signature σ issued by a trusted third party, i.e. $C = \{A, \sigma\}$. Users can optionally share A to prove identity and permissions.
- *Verifiable credential (VC).* A VC is a set of claims issued by an issuer. $VC = \{A, \sigma_{issuer}\}$ contains the set of attributes A and the digital signature of the issuer σ_{issuer}, the user can selectively disclose some of the attributes $A' \subseteq A$ to prove specific identity information.

TABLE I
Related Work Involving Current Work(Fully: ✓, Partial: ∗, Not Applicable: ×).

Literature	DID + TLS	VC	Issuer Option
Perugini *et al.* [17]	✓	✓	×
Garzon *et al.* [18]	✓	✓	×
Urien [19]	∗	×	×
ČUČKO *et al.* [20]	∗	✓	✓
Bahar *et al.* [21]	✓	×	×
Beckwith *et al.* [22]	✓	×	×
Chen *et al.* [23]	✓	×	×
Mühle *et al.* [24]	∗	✓	✓
This work	✓	✓	✓

C. Integration of TLS and DID

The emergence of DID and VC has introduced a new paradigm for identity authentication. Unlike centralized certificate authority, DIDs utilize distributed ledger technology for decentralized authentication, mitigating single points of failure and enhancing attack resistance. This approach allows individuals to autonomously manage their digital identities across various systems and scenarios. There exist works have been conducted for Self Sovereign Identity (SSI)/TLS, VC, Issuer Option and Range of Application aspects as shown in Table I. Combining theoretical analyses and empirical studies, the integration of DID into the TLS protocol can enhance the security and trust of the authentication mechanism. Unlike traditional X.509 certificates, which require issuance and management by authoritative organizations, DIDs enable users to autonomously manage and control their identities, reducing reliance on third parties.

III. Problem Description

A. System Model

The new model, as provided in Fig. 3, utilises the Blockchain as an Identifier Registry for a traditional identity management system and allows individuals to have their own digital identities by filtering the Committee to act as a certificate issuer and considering the Server to act as a verifier of the identity.

Fig. 3. The TLS-D framework.

B. Design Goals

We aim to achieve the following three design goals:

- *Controllability.* Allows users to manage and issue their own identity credentials, supporting selective disclosure of identity information. This decentralized trust model reduces reliance on a single trust authority, thereby enhancing system security and flexibility.
- *Security.* The integration of a decentralized identity system with the TLS protocol must meet pre-existing security requirements, including data protection and authentication integrity.
- *Compatibility.* The integration solution must be compatible with existing TLS protocol standards while supporting the specifications of decentralized identity systems.

IV. Detailed Design

A. Protocol Overview

TLS-D consists of three main phases, namely, key exchange, server parameters, and authentication. Below, we describe the workflow of these phases shown in Fig. 4. **i) Key exchange.** In this phase, the client interacts with the Identifier Registry to register a specific decentralized identity for the user, upload it to the blockchain for future on-chain credential verification (Steps ① and ② in Fig. 4), and the Committee will authenticate its identity and generate the appropriate declaration and signature (Steps ③ - ⑥). Finally, the client hello message is initiated (step ⑦). **ii) Server parameters.** Establish other handshake parameters (step ⑦ and ⑩, whether the client is authenticated, application-layer protocol support, etc.). **iii) Authentication.** In this phase, the server will request certain identity proofs from the Identifier Registry (steps ⑧ and ⑨), verify them and return the encrypted information and private information such as identity verification to the client to complete this handshake (step ⑩).

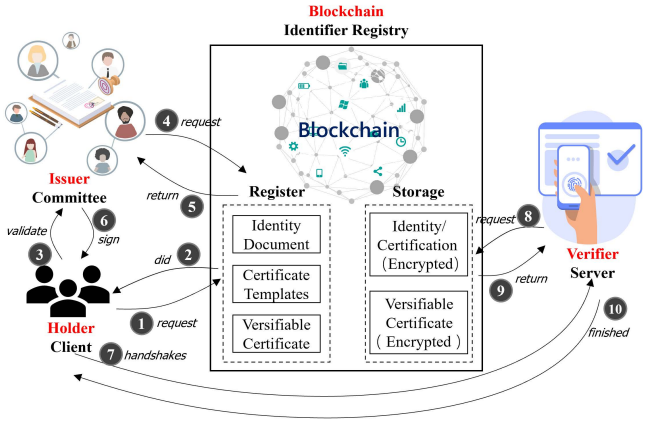

Fig. 4. TLS-D workflow.

B. Protocol Details

Our goals are threefold, *(i)* to design a new key exchange mechanism that allows the client to request a DID identity and complete CA authentication, *(ii)* to maintain interoperability with public-key certificates, i.e., to support a hybrid handshake with certificates and VC, and to fall back to the original handshake, and *(iii)* to retain all the security features of TLS 1.3. Fig. 5 shows the message flow of the new extension using the new handshake protocol. The protocol uses a new TLS-D that allows the client and server to negotiate the VC authentication model and common blockchain methods as identifier registry. the client sends this extension ClientHello to initiate the DID handshake. The server can also request identifier registry authentication via VC/VP in the extended field.

TLS -D authentication does not rely on internet protocol (IP) or domain name system (DNS), but instead depends on public key infrastructure (PKI) and certificate chains. The client wraps the VC into a VP and signs it before presenting it to the server for authentication.The DID handshake captures this design principle with two new messages:The VC message carries the VC/VP and DIDVerify messages, which are signatures performed on all messages, and the VC carries the CA and client private key. This design choice reduces the total

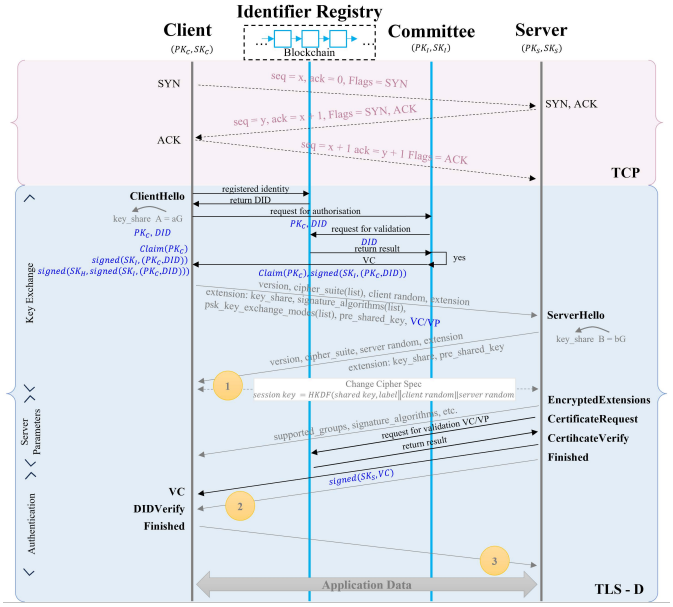

Fig. 5. Message flow in the TLS-D handshake protocol.

number of signatures and signature verifications during the handshake while maintaining compliance with the VP concept.

The client receives the DIDVerify message, checks that the VC conforms to the scheme specified in the @context field, checks the validity of the VC metadata, verifies the signature of the issuer on the VC, and then extracts and parses the DID from the VC's credentialSubject field and retrieves the server public key from the blockchain.

Finally, the Client verifies the signature in the DIDVerify message. Note that both Client and server maintain lists of public keys of their trusted issuers.Client authenticates with server in the same way.

1) Key Exchange: The original idea of decentralized identity assumes that users themselves issue and manage their own ids (mainly identifiers). However, in reality, it is not common for individuals to build and manage their own servers, and a separate entity must issue identities. Therefore, adding two additional identities, i.e. Committee, Identifier Registry. The decentralized identity will be issued and verified by the issuer (Committee) of the credentials associated with the ID and the registry that registers the DID and uses the credential template (Identifier Registry).

The structure of the standard ClientHello message has been extended to include custom extension types (e.g., $extensionType = 0x002b$). This extension payload employs TLV (Type-Length-Value) encoding and contains the client's DID identifier, the declared Verifiable Credential (VC) template ID, and a digital signature based on the client's private key for the current handshake random (ClientRandom). This signature ensures the request's freshness, effectively preventing replay attacks.

2) Server Parameters and Authentication: The client verifies the identity of the server through a certificate provided by the server to ensure communication with the intended server; the server in turn verifies the identity of the client through a certificate provided by the client (if client certificates are enabled), thus increasing security. While TLS 1.3 improves security, it still relies on the trustworthiness of the certificate authority, and if the certificate authority is breached, the entire chain of trust is at risk.

Implement server-side two-factor authentication algorithms, including traditional certificate verification and DID credential verification. **The focus is on DID credential verification:** First, parse the DID identifier from the extended data and retrieve its corresponding DID document via a decentralized resolver. Extract the master public key from this document for signature verification. Subsequently, use this public key to validate the signature in ClientHello. Upon successful signature verification, the server queries and validates the verifiable credential (VC) provided by the client based on the declared template ID. This validation includes checking the VC's issuer signature, validity period, and whether its declared attributes comply with the access control policy. Only after both verifications succeed will the handshake proceed. The master secret of the final session is derived using both traditional key exchange material and the DID verification success confirmation message, thereby cryptographically binding the DID identity to the subsequent encrypted channel established.

3) Committee Selection: In the TLS 1.3 protocol, the selection of adopting committee nodes is a key process to ensure the security, stability and fairness of the network. This process can be designed using a Distributed Key Generation (DKG) protocol to ensure that the selection of committee nodes is both secure and decentralized, and that the integrity of the process is maintained through encryption and consensus building mechanisms.

- *Pre-selection Phase*. All candidate nodes participate in the DKG process. Using threshold cryptography, private key fragments are distributed to ensure that no single node holds the entire key, enhancing security and decentralization.
- *Consensus Building*. The DKG protocol enables consensus by requiring a threshold of cooperating nodes to unlock cryptographic keys and perform operations. This ensures that only nodes with demonstrated reliability and stability participate in the final committee, strengthening the integrity of the selection process.
- *Decentralized Filtering*. DKG ensures each node holds only a key fragment, preventing any single node from altering results. This decentralizes the filtering process, eliminating the need for central authority and enhancing the transparency and fairness of node selection.

V. Experiments

A. Implementation

In our experiments, we used Visual Studio 2022, which supports the TLS 1.3 protocol, to implement a test demo of TLS that tested TLS 1.3 integrated connectivity, including integration with blockchain for decentralized identity management. We consider simulated client requests for URLs to

test the TLS 1.3 protocol, including authentication, exception catching, security checking and complexity testing, as well as a visual representation of response times. It effectively demonstrated how to combine network requests, cryptography, and blockchain interactions in a user-friendly Windows Forms application. We effectively tested and compared the TLS 1.3 performance and security of different servers, recording and displaying server response times, certificate information and connection details.

B. Results and Analysis

We collected and plotted performance metrics for response time during the optimization process, but the data may vary due to changing network status, as shown in Fig. 6. In terms of TLS protocols, by using the DID infrastructure, the DecentralizedIdentity class is utilized for credential management, the BlockchainCommittee class is used to enforce the decentralized trust DKG protocol, and the IdentifierRegistry is used to manage approved credential templates. URLs supporting TLS 1.3 were tested URLs [1] [2].

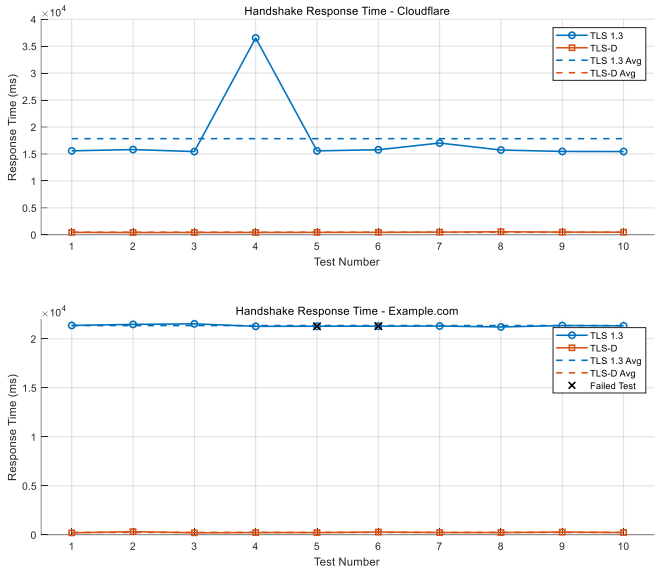

Fig. 6. Handshake response time comparison between TLS 1.3 and TLS-D protocols.

As shown in table II, the comparative analysis reveals dramatic latency differences between TLS 1.3 and TLS-D protocols. For Cloudflare connections, TLS-D demonstrates 97.1% reduction in average handshake time (473.71 ms vs. 17839.01 ms) compared to TLS 1.3. The performance gap is even more pronounced in maximum latency scenarios, where TLS 1.3 exhibits extreme spikes up to 36520.99 ms - 64.3× longer than TLS-D's peak of 568.00 ms. The Example test not only maintains 100% success rate (80% for TLS 1.3), but also reduces the minimum latency by 105× (21205.79 ms → 200.70 ms).

[1]https://www.cloudflare.com,
[2]https://www.example.com

TABLE II
HANDSHAKE RESPONSE TIME COMPARISON BETWEEN TLS 1.3 AND TLS-D PROTOCOLS.

Protocol	URL[1]		URL[2]	
	Response (ms)	Statistics	Response (ms)	Statistics
TLS 1.3	15580.13 15816.45 15444.97 36520.99 15576.43 15779.34 17028.36 15726.68 15467.98 15448.80	Avg: 17839.01 Min: 15444.97 Max: 36520.99 SD: 6243.05 Success: 100%	21360.86 21468.66 21525.75 21268.07 – – 21299.18 21205.79 21350.19 21321.99	Avg: 21350.06 Min: 21205.79 Max: 21525.75 SD: 97.44 Success: 80%
TLS-D	458.79 424.54 434.70 440.99 454.97 466.29 501.30 568.00 501.82 485.72	Avg: 473.71 Min: 424.54 Max: 568.00 SD: 40.28 Success: 100%	200.70 324.34 206.19 230.16 225.81 279.54 232.74 230.11 277.26 233.29	Avg: 244.01 Min: 200.70 Max: 324.34 SD: 36.17 Success: 100%

An experimental approach was used to evaluate the security performance of the 0-RTT feature of the TLS-D protocol. Baseline measurements are first established by performing the standard TLS 1.3 handshake process as a control group and recording the complete handshake time as baseline data. Subsequently, attack simulation experiments are conducted to simulate 0-RTT replay attack scenarios by initiating five (n=5) consecutive handshake attempts using the exact same session parameters under the same network environment and system configuration, with the handshake establishment elapsed time (in milliseconds) accurately recorded for each experiment, with the results shown in Fig. 7. The statistical difference between the initial handshake time μ_{initial} and the average time of the replay handshake μ_{replay} is compared, while the percentage time reduction $\Delta T = \frac{\mu_{\text{initial}} - \mu_{\text{replay}}}{\mu_{\text{initial}}} \times 100\%$ is calculated as an indicator of performance improvement. With this controlled experimental design, we are able to quantitatively assess: 1) the theoretical performance gain of the 0-RTT mode, 2) the temporal characteristics of the replay attack, and 3) the ability of the protocol implementation to recognize repeated connections.

The experimental evaluation under 0-RTT replay attack conditions demonstrates significant performance enhancements in the improved protocol, as detailed in Table III. For URL[1], handshake latency was reduced from 15,501.1 ms and 15,437.7 ms in the conventional implementation to 521.6 ms and 466.9 ms, representing dramatic reductions of 96.6% and 97.0% respectively. These improvements correspond to speedup factors of 29.7× for initial handshakes and 33.1× for average handshake duration. The performance gains were even more pronounced for URL[2], where initial handshake

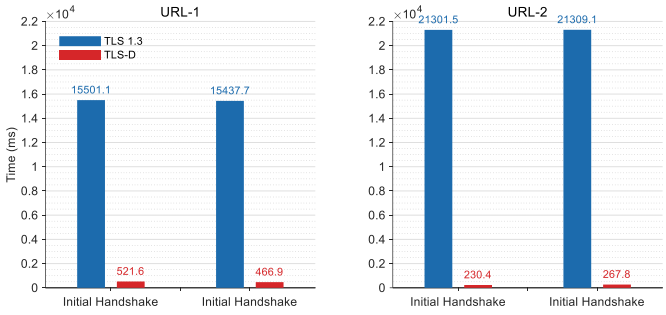

Fig. 7. Performance comparison under 0-RTT replay attack scenario.

TABLE III
PERFORMANCE COMPARISON UNDER 0-RTT REPLAY ATTACK SCENARIO.

URL	Metric	TLS 1.3	TLS-D
URL[1]	Initial Handshake	15501.1 ms	521.6 ms
	Average Handshake	15437.7 ms	466.9 ms
	Reduction	Initial: 96.6%	Average: 97.0%
	Speedup Factor	Initial: 29.7×	Average: 33.1×
URL[2]	Initial Handshake	21301.5 ms	230.4 ms
	Average Handshake	21309.1 ms	267.8 ms
	Reduction	Initial: 98.9%	Average: 98.7%
	Speedup Factor	Initial: 92.5×	Average: 79.6×

latency decreased from 21,301.5 ms to just 230.4 ms (98.9% reduction), with average latency improving from 21,309.1 ms to 267.8 ms (98.7% reduction). The results illustrate a significant speedup in the TLS-D response rate, with almost an order of magnitude improvement compared to the baseline protocol.

All measurements exhibited exceptional stability with sub-millisecond variance ($\sigma < 1ms$) across both test cases, confirming the robustness of the enhancements. The consistent magnitude of improvement across different URL configurations demonstrates the protocol's generalized performance benefits under 0-RTT replay attack conditions.

TABLE IV
PACKET CAPTURE EVALUATION: COMPARATIVE ANALYSIS.

Package Type	TLS 1.3 (Byte)	TLS-D (Byte)	Amount	Rate
ClientHello	259	280	+21	+8.1%
ServerHello	128	136	+8	+6.3%
Certificate	1940	1802	-138	-7.1%
Finished	53	56	+3	+5.7%
ApplicationData	111	108	-3	-2.7%

As shown in table IV, through two independent packet capture evaluations, TLS-D exhibited considerable optimization of certificate payloads, decreasing from 1940 bytes to 1802 bytes, representing a reduction of 7.1%, all while ensuring stability in performance and consistency in protocol. This suggests the possible adoption of measures to streamline the certificate chain. Meanwhile, the size of the ClientHello message increased by 21 bytes (from 259 bytes to 280 bytes), which corresponds to the incorporation of the integrated DID extension field, indicating enhanced authentication mechanisms. Security assessments demonstrate that enhanced security has been achieved while maintaining high performance. DID extensions provide robust authentication capabilities, while certificate size optimization reduces transmission overhead while preserving verification integrity.

C. Discussion

In the initial testing phase, large public service providers were selected as test endpoints to rapidly validate the compatibility of the protocol with existing internet services. This approach introduced uncontrollable variables such as external network latency (approximately 30ms RTT) and server processing queue delays, which contributed to the observed 65ms handshake time. To precisely measure the inherent overhead introduced by protocol enhancements, we plan to design strictly controlled experiments. Customized TLS 1.3 servers and clients will be deployed within an isolated LAN experimental environment.

VI. CONCLUSION AND FUTURE WORK

Combining blockchain technology for decentralized identity registration adds an important security dimension to the TLS 1.3 application scenarios. Through DID registration and its signature mechanism, we are able to ensure the authenticity and immutability of the identity, thus providing a higher level of security for users. Future research can focus on advancing the establishment of trusted digital identities through decentralized identifiers and ensuring their authenticity across various application scenarios. In addition, exploring modifications to the certificate chain to enhance its reliability and verifiability will be crucial to bolstering the security of the entire trust framework.

ACKNOWLEDGMENT

We thank the reviewers for their insightful comments. This work was supported in part by National Key R&D Project of China (No. 2023YFB3308501).

REFERENCES

[1] R. O. Yldz and A. Ylmazer-Metin, "Design and implementation of tls accelerator," in *2022 IEEE 15th Dallas Circuit And System Conference (DCAS)*, 2022, pp. 1–4.

[2] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/info/rfc8446

[3] S. Sivakorn, A. D. Keromytis, and J. Polakis, "That's the way the cookie crumbles: Evaluating https enforcing mechanisms," in *Proceedings of the 2016 ACM on Workshop on Privacy in the Electronic Society*, ser. WPES '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 7181. [Online]. Available: https://doi.org/10.1145/2994620.2994638

[4] G. Arfaoui, X. Bultel, P.-A. Fouque, A. Nedelcu, and C. Onete, "The privacy of the tls 1.3 protocol," *PoPETs*, vol. 2019, no. 4, pp. 190–210, 2019. [Online]. Available: https://doi.org/10.2478/popets-2019-0065

[5] S. Chen, S. Jero, M. Jagielski, A. Boldyreva, and C. Nita-Rotaru, "Secure communication channel establishment: Tls 1.3 (over tcp fast open) versus quic," *Journal of Cryptology*, vol. 34, no. 3, p. 26, 2021. [Online]. Available: https://doi.org/10.1007/s00145-021-09389-w

[6] C. Cremers, M. Horvat, S. Scott, and T. van der Merwe, "Automated analysis and verification of tls 1.3: 0-rtt, resumption and delayed authentication," in *2016 IEEE Symposium on Security and Privacy (SP)*, 2016, pp. 470–485.
[7] B. Dowling, M. Fischlin, F. Gnther, and D. Stebila, "A cryptographic analysis of the tls 1.3 handshake protocol," *Journal of Cryptology*, vol. 34, no. 4, p. 37, 2021. [Online]. Available: https://doi.org/10.1007/s00145-021-09384-1
[8] P. Kotzias, A. Razaghpanah, J. Amann, K. G. Paterson, N. Vallina-Rodriguez, and J. Caballero, "Coming of age: A longitudinal study of tls deployment," in *Proceedings of the Internet Measurement Conference 2018*, ser. IMC '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 415428. [Online]. Available: https://doi.org/10.1145/3278532.3278568
[9] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe, "A comprehensive symbolic analysis of tls 1.3," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 17731788. [Online]. Available: https://doi.org/10.1145/3133956.3134063
[10] H. Lee, D. Kim, and Y. Kwon, "Tls 1.3 in practice:how tls 1.3 contributes to the internet," in *Proceedings of the Web Conference 2021*, ser. WWW '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 7079. [Online]. Available: https://doi.org/10.1145/3442381.3450057
[11] K. Bhargavan, B. Blanchet, and N. Kobeissi, "Verified models and reference implementations for the tls 1.3 standard candidate," in *2017 IEEE Symposium on Security and Privacy (SP)*, 2017, pp. 483–502.
[12] G. Zyskind, O. Nathan, and A. S. Pentland, "Decentralizing privacy: Using blockchain to protect personal data," in *2015 IEEE Security and Privacy Workshops*, 2015, pp. 180–184.
[13] T. Gu, M. Han, S. He, and X. Chen, "Trap contract detection in blockchain with improved transformer," in *GLOBECOM 2023 - 2023 IEEE Global Communications Conference*, 2023, pp. 5141–5146.
[14] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.
[15] M. Bartoletti and L. Pompianu, "An empirical analysis of smart contracts: Platforms, applications, and design patterns," in *Financial Cryptography and Data Security*, M. Brenner, K. Rohloff, J. Bonneau, A. Miller, P. Y. Ryan, V. Teague, A. Bracciali, M. Sala, F. Pintore, and M. Jakobsson, Eds. Cham: Springer International Publishing, 2017, pp. 494–509.
[16] S. He, T. Sun, Q. Tang, C. Wu, N. Lipka, C. Wigington, and R. Jain, "Secure and efficient agreement signing atop blockchain and decentralized identity," in *Blockchain and Trustworthy Systems*, D. Svetinovic, Y. Zhang, X. Luo, X. Huang, and X. Chen, Eds. Singapore: Springer Nature Singapore, 2022, pp. 3–17.
[17] L. Perugini and A. Vesco, "On the integration of self-sovereign identity with TLS 1.3 handshake to build trust in iot systems," *CoRR*, vol. abs/2311.00386, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2311.00386
[18] S. R. Garzon, D. Natusch, A. Philipp, A. Kpper, H. J. Einsiedler, and D. Schneider, "Did link: Authentication in tls with decentralized identifiers and verifiable credentials," 2024. [Online]. Available: https://arxiv.org/abs/2405.07533
[19] P. Urien, "A new iot trust model based on tls-se and tls-im secure elements: A blockchain use case," in *2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC)*, 2021, pp. 1–2.
[20] . uko and M. Turkanovi, "Decentralized and self-sovereign identity: Systematic mapping study," *IEEE Access*, vol. 9, pp. 139 009–139 027, 2021.
[21] B. Houtan, A. S. Hafid, and D. Makrakis, "A survey on blockchain-based self-sovereign patient identity in healthcare," *IEEE Access*, vol. 8, pp. 90 478–90 494, 2020.
[22] E. Beckwith and G. Thamilarasu, "Ba-tls: Blockchain authentication for transport layer security in internet of things," in *2020 7th International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*, 2020, pp. 1–8.
[23] J. Chen, S. Yao, Q. Yuan, K. He, S. Ji, and R. Du, "Certchain: Public and efficient certificate audit based on blockchain for tls connections," in *IEEE INFOCOM 2018 - IEEE Conference on Computer Communications*, 2018, pp. 2060–2068.
[24] A. Mhle, A. Grner, T. Gayvoronskaya, and C. Meinel, "A survey on essential components of a self-sovereign identity," *Computer Science Review*, vol. 30, pp. 80–86, 2018.